Ministerul Educaţiei şi Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică şi Microelectronică
Departamentul Ingineria Software şi Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 1

The OS as a Resource Manager

Track A: The Real OS, from the Outside (Linux)

Done by:
st. gr. FAF-241
Serghei Barladean

Checked by:
Patricia Reitman

Chișinău – 2026

Contents

1. Introduction.....3
2. System Information.....3
3. Files and Directories.....3
3.1 Observations.....4
3.2 Interesting Command Output.....4
4. Processes.....5
4.1 Observations.....5
4.2 Interesting Command Output.....5
5. Memory Management.....6
5.1 Observations.....6
5.2 Interesting Command Output.....7
6. Devices and Storage.....7
6.1 Observations.....7
6.2 Interesting Command Output.....8
7. Conclusion.....8

1. Introduction

The purpose of this laboratory work is to observe how an operating system manages the main resources of a computer system.

The experiment was performed on Ubuntu 24.04 running in a Multipass virtual machine on macOS (Apple Silicon, aarch64). The Linux virtual machine was used because macOS has no /proc filesystem and uses a different set of system tools. During the laboratory work, four major areas of operating system resource management were investigated: files and directories, processes, memory, and devices/storage.

2. System Information

The following commands were used to identify the current user, operating system kernel, and system uptime:

```
whoami
uname -a
uptime
```

The system reported the following information:

```
ubuntu
Linux os 6.8.0-142-generic #142-Ubuntu SMP PREEMPT_DYNAMIC
Wed Sep  2 14:30:29 UTC 2026 aarch64 aarch64 aarch64 GNU/Linux
23:12:07 up 1 min,  2 users,  load average: 0.04, 0.02, 0.00
```

The Linux environment was running using the standard Ubuntu generic Linux kernel.

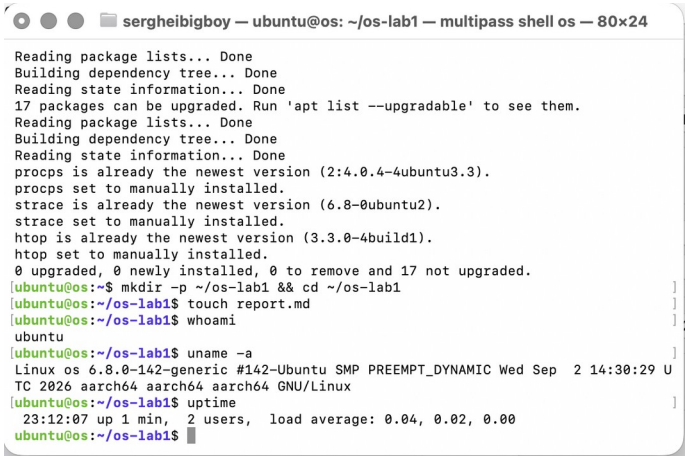


Figure 1. Terminal output of whoami, uname -a and uptime

3. Files and Directories

In the first part of the laboratory work, the Linux file system was explored and basic file-management operations were performed.

Commands such as the following were used:

```
pwd
ls -la /
ls -la ~
cd /etc
ls | head
```

A demonstration directory and several files were then created:

```
mkdir demo
cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
```

3.1. Observations

1. The files created during the experiment were owned by user ubuntu and belonged to group ubuntu.
2. After executing `chmod 600 note.txt` the file permissions became `-rw-------`. The first character, `-`, indicates that note.txt is a regular file. The next three characters, `rw-`, indicate that the owner can read and write the file but cannot execute it. The following `---` indicates that members of the group have no permissions, while the final `---` indicates that other users also have no permissions.
3. Two directories directly under the root directory / are:
    - `/etc` – contains system-wide configuration files.
    - `/home` – contains users' home directories.

3.2. Interesting Command Output

The command ls -l displayed the following result before and after the permissions were changed:

```
total 8
-rw-rw-r-- 1 ubuntu ubuntu 24 Oct  1 23:13 note.txt
-rw-rw-r-- 1 ubuntu ubuntu 24 Oct  1 23:13 renamed.txt

-rw------- 1 ubuntu ubuntu 24 Oct  1 23:13 note.txt   (after chmod
600)
```

This output demonstrates that Linux stores information about ownership, permissions, size, and modification time for every file. The data of note.txt stayed the same, while the operating system removed access for the group and for other users.

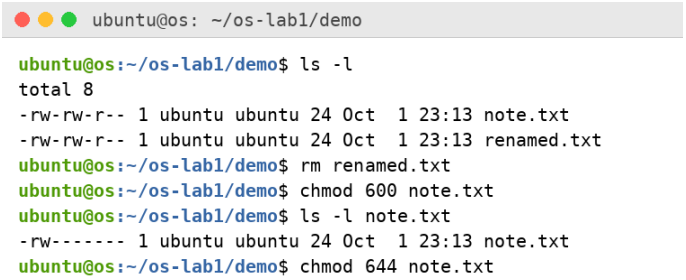


Figure 2. File listing and permissions before and after chmod 600

4. Processes

The second part focused on process management.

The currently running processes were inspected using:

```
ps aux | head
ps aux | wc -l
top
```

A new process was created in the background:

```
sleep 300 &
```

It was inspected using:

```
jobs
ps -ef | grep sleep
```

The process was then terminated using the kill command. The Linux pseudo-filesystem /proc was also used to inspect information maintained by the kernel about the process.

4.1. Observations

1. Process number 1 has PID 1. On the tested system it was systemd. This is the init process, started first by the kernel, which is responsible for starting and managing system services and is the ancestor of all other user-space processes.
2. Approximately 101 processes were running while the virtual Linux environment was mostly idle (ps aux | wc -l printed 102, one line of which is the header).
3. For the test sleep process, the file `/proc/1924/status` contained: `State: S (sleeping)`. This means that the process existed but was waiting rather than actively using CPU time.

4.2. Interesting Command Output

The process status contained the following information:

```
Name:   sleep
State:  S (sleeping)
Pid:    1924
PPid:   1571
VmPeak: 5684 kB
VmSize: 5684 kB
```

PPid 1571 is the bash shell that started the sleep process, which shows the parent-child process tree maintained by the operating system. The output also shows that the OS keeps detailed information about every process and exposes part of it through the /proc filesystem.

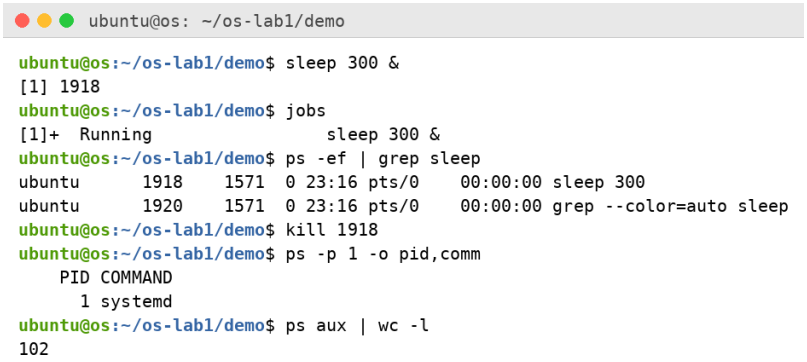


Figure 3. Background process, its PID, parent and PID 1

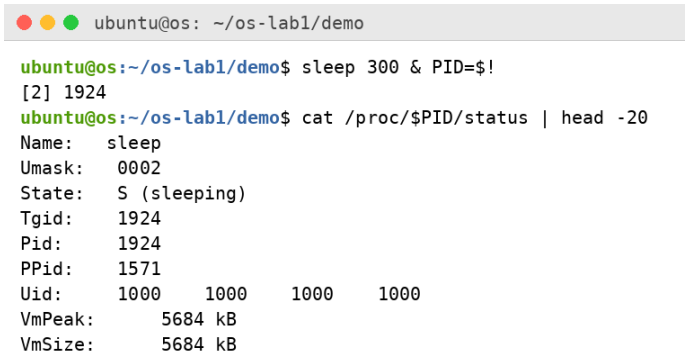


Figure 4. Process information exposed by the kernel in /proc

5. Memory Management

The third part of the laboratory work investigated how Linux manages physical memory and swap space.

The following commands were executed:

```
free -h
cat /proc/meminfo | head -6
```

The resident memory used by a simple sleep process was inspected with:

```
grep VmRSS /proc/$PID/status
```

5.1. Observations

1. The system had approximately 3.8 GiB of total RAM (MemTotal 3995108 kB), with about 3.1 GiB free at the time of observation and 3.5 GiB available.
2. Swap is storage space on disk that can be used by the operating system as an extension of physical memory when RAM is under pressure. The tested system had 0 B of swap configured, so none was available.
3. The sleep process used approximately 1760 kB (about 1.7 MB) of resident memory according to its VmRSS value. This amount is small, which is expected because the sleep

process performs almost no computation, but it is not zero: even a program that does nothing needs memory for its code, the C library and its stack. It is also smaller than its virtual size (VmSize 5684 kB), because only the pages that the process actually uses are kept in RAM.

5.2. Interesting Command Output

```
               total  used   free  shared  buff/cache  available
Mem:           3.8Gi  275Mi  3.1Gi  1.1Mi  618Mi       3.5Gi
Swap:          0B     0B     0B
```

The output demonstrates how the operating system divides physical memory between currently used memory, free memory, and cached data. Much of the memory that is not listed as free is used as buff/cache: the OS uses spare RAM to cache files and returns it to programs when needed.

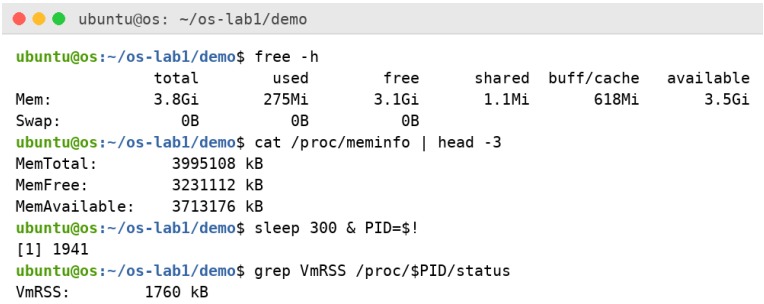


Figure 5. Total, used and free memory, swap, and VmRSS of a sleep process

6. Devices and Storage

The final experimental part focused on storage devices, mounted filesystems, and device interfaces.

The following commands were used:

```
df -h
lsblk
du -sh ~/os-lab1
ls -l /dev | head
mount | head
```

6.1. Observations

1. The root filesystem / was mounted on: `/dev/sda1` (partition 1 of the disk sda, filesystem type ext4, 19 GB in size and 11% used).
2. One example of a device interface found inside /dev is `/dev/console`. It represents the system console and provides a file-like interface for terminal input and output. Another example is `/dev/sda`, which stands for the whole 20 GB virtual disk.
3. The UNIX principle "everything is a file" means that many operating system resources, including hardware devices, are exposed through file-like interfaces that programs can access using common read and write operations.

6.2. Interesting Command Output

The lsblk command showed the block devices visible to Linux:

```
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda       8:0    0   20G  0 disk
├─sda1    8:1    0   19G  0 part /
├─sda15   8:15   0   99M  0 part /boot/efi
└─sda16 259:0    0  923M  0 part /boot
```

The output shows that one physical disk (sda) is split into partitions: sda1 contains the root filesystem, sda15 is the EFI system partition and sda16 holds /boot. Each partition is mounted at its own place in the single directory tree.

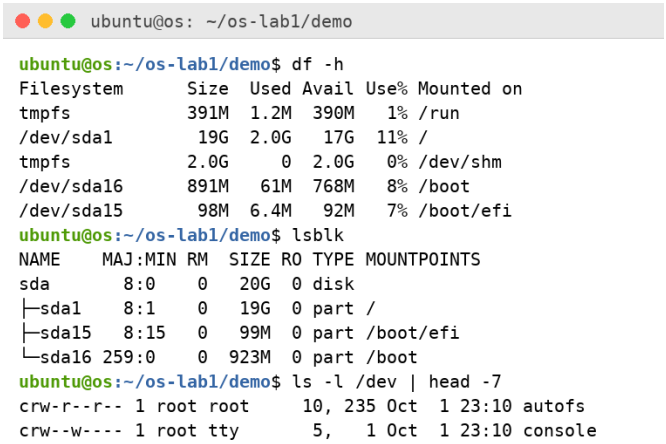


Figure 6. Mounted filesystems, block devices and entries from /dev

7. Conclusion

The operating system manages four major categories of resources: files, processes, memory, and devices/storage.

File management was observed using commands such as ls -l; process management was inspected using ps aux; memory usage was examined using free -h; and storage devices were inspected using lsblk.

The laboratory work demonstrated how Linux provides structured and controlled access to system resources and acts as an intermediary between running programs and the underlying hardware.